

Laboratorio de Computación

Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 11: Funciones

Integrante(s): Nolasco Ramírez Sofia

Macay Martínez David

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 15 de mayo de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:

El alumnado elaborará programas en C donde la solución del problema se divida en funciones. Distinguirá lo que es el prototipo o firma de una función y la implementación de ella, asimismo manipulará parámetros tanto en la función principal como en otras.

Introducción:

La escritura en un solo bloque de código es ineficiente para el mantenimiento, la depuración y la colaboración, para el desarrollo de software profesional. La programación modular es una solución a esta problemática. Esto se logra mediante el uso de funciones, los cuales son bloques de código diseñados para realizar una tarea específica y que pueden ser invocados múltiples veces dentro del programa.

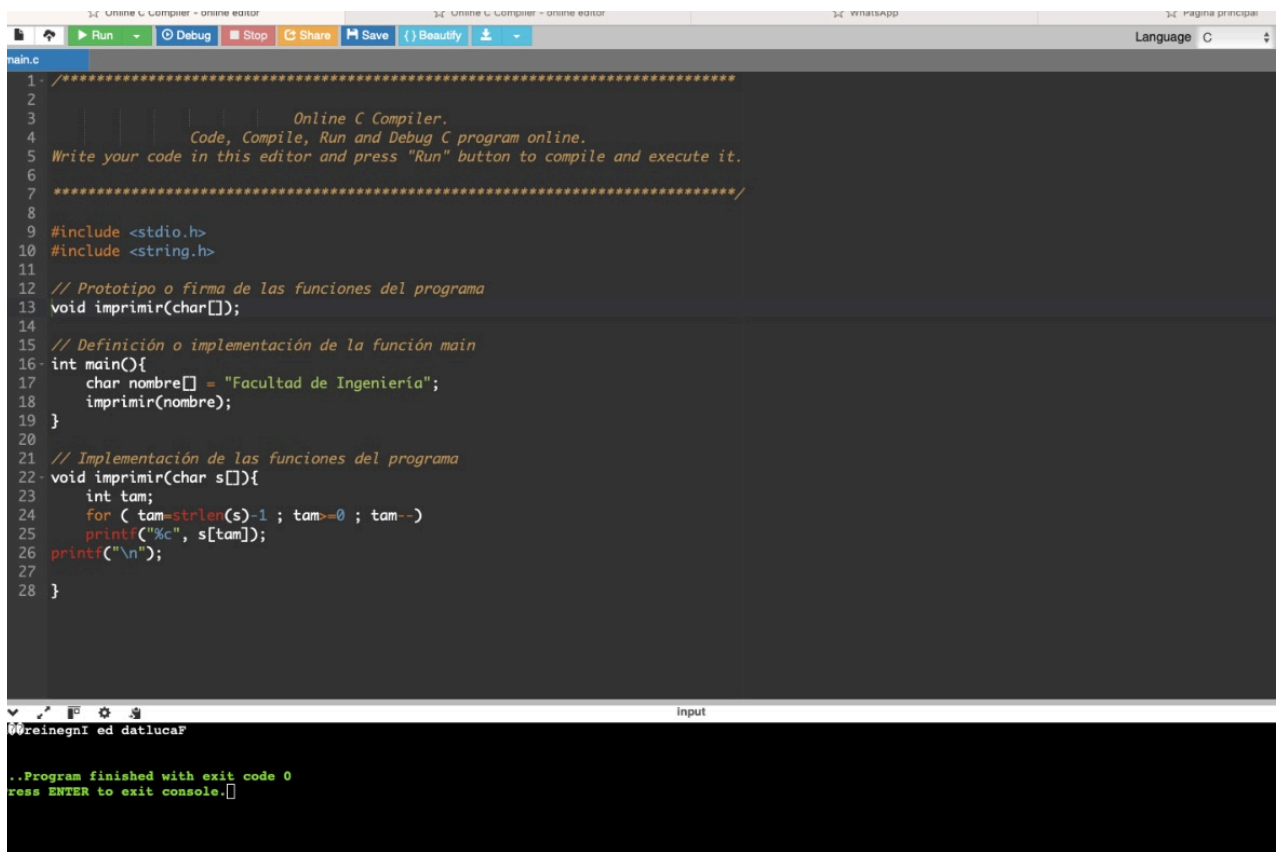
En el desarrollo de esta práctica es importante la manipulación de parámetros, los cuales funcionarán como canal de comunicación entre funciones, permitiendo el paso de información de entrada y el retorno de resultados. La comprensión y dominio de parámetros, además de su flujo, son importantes para crear un código reutilizable, organizado y con una estructura lógica clara, deben considerarse para cualquier proyecto de ingeniería.

Desarrollo:

Programa 1.c

El siguiente programa contiene dos funciones: la función main y la función imprimir. La función main manda llamar a la función imprimir. La función imprimir recibe como parámetro un arreglo de caracteres y lo recorre de fin a inicio imprimiendo cada carácter del arreglo.

Dentro del código strlen es una función que recibe como parámetro un arreglo de caracteres y regresa como valor de retorno un entero que indica la longitud de la cadena. La función se encuentra dentro de la biblioteca string.h, por eso se incluye ésta al principio del programa.



```
1 // *****
2
3           Online C Compiler.
4           Code, Compile, Run and Debug C program online.
5 Write your code in this editor and press "Run" button to compile and execute it.
6
7 // *****
8
9 #include <stdio.h>
10 #include <string.h>
11
12 // Prototipo o firma de las funciones del programa
13 void imprimir(char[]);
14
15 // Definición o implementación de la función main
16 int main(){
17     char nombre[] = "Facultad de Ingeniería";
18     imprimir(nombre);
19 }
20
21 // Implementación de las funciones del programa
22 void imprimir(char s[]){
23     int tam;
24     for ( tam=strlen(s)-1 ; tam>=0 ; tam-- )
25         printf("%c", s[tam]);
26     printf("\n");
27 }
28 }
```

Input

reinegni ed datlucaF

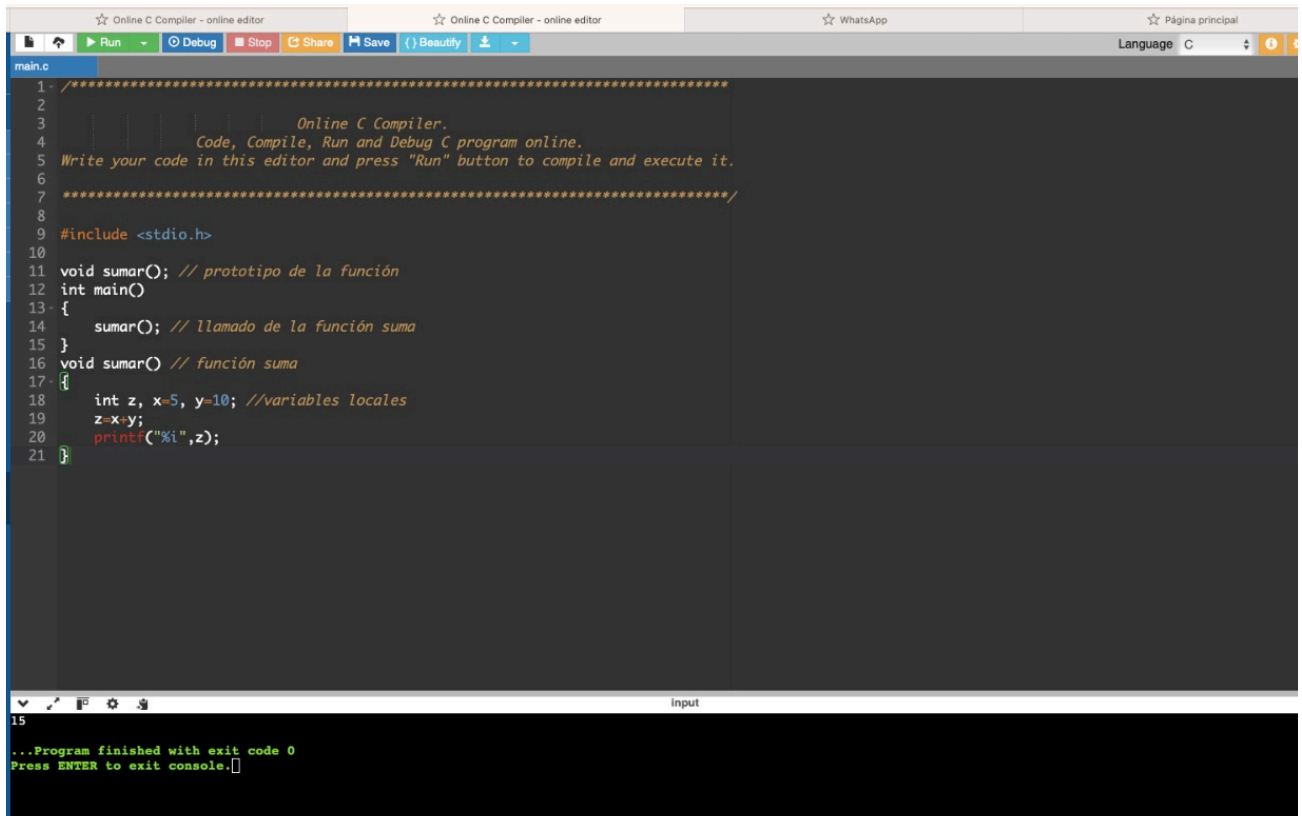
..Program finished with exit code 0
ress ENTER to exit console.

Figura 1: Código con 2 funciones

Programa 2.c

El siguiente programa muestra la declaración y uso de variables locales en la función de suma.

Las variables que se declaran fuera de cualquier función se llaman variables globales. Las variables globales existen durante la ejecución de todo el programa y pueden ser utilizadas por cualquier función.



```
1  //*****
2
3      Online C Compiler.
4      Code, Compile, Run and Debug C program online.
5      Write your code in this editor and press "Run" button to compile and execute it.
6  //*****
7
8
9  #include <stdio.h>
10
11 void sumar(); // prototipo de la función
12 int main()
13 {
14     sumar(); // llamado de la función suma
15 }
16 void sumar() // función suma
17 {
18     int z, x=5, y=10; //variables locales
19     z=x+y;
20     printf("%i",z);
21 }
```

Input

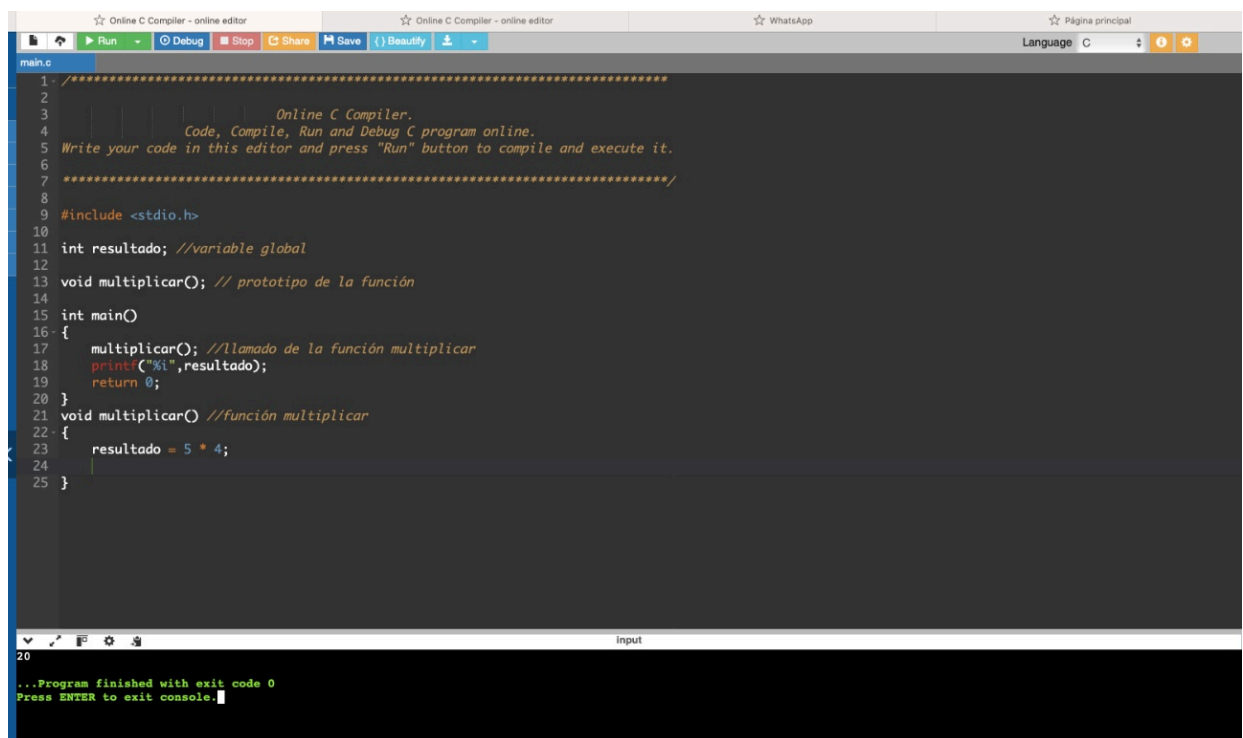
15

...Program finished with exit code 0
Press ENTER to exit console.

Figura 2: Código con variables locales

Programa 3.c

El siguiente programa muestra la declaración de la variable global resultado, la cual es utilizada en ambas funciones. Las variables globales en C se inician, por defecto, con el valor de 0.



```
1  //*****
2
3      Online C Compiler.
4      Code, Compile, Run and Debug C program online.
5      Write your code in this editor and press "Run" button to compile and execute it.
6  //*****
7
8
9  #include <stdio.h>
10
11 int resultado; //variable global
12
13 void multiplicar(); // prototipo de la función
14
15 int main()
16 {
17     multiplicar(); //llamado de la función multiplicar
18     printf("%i",resultado);
19     return 0;
20 }
21 void multiplicar() //función multiplicar
22 {
23     resultado = 5 * 4;
24 }
25 }
```

Input

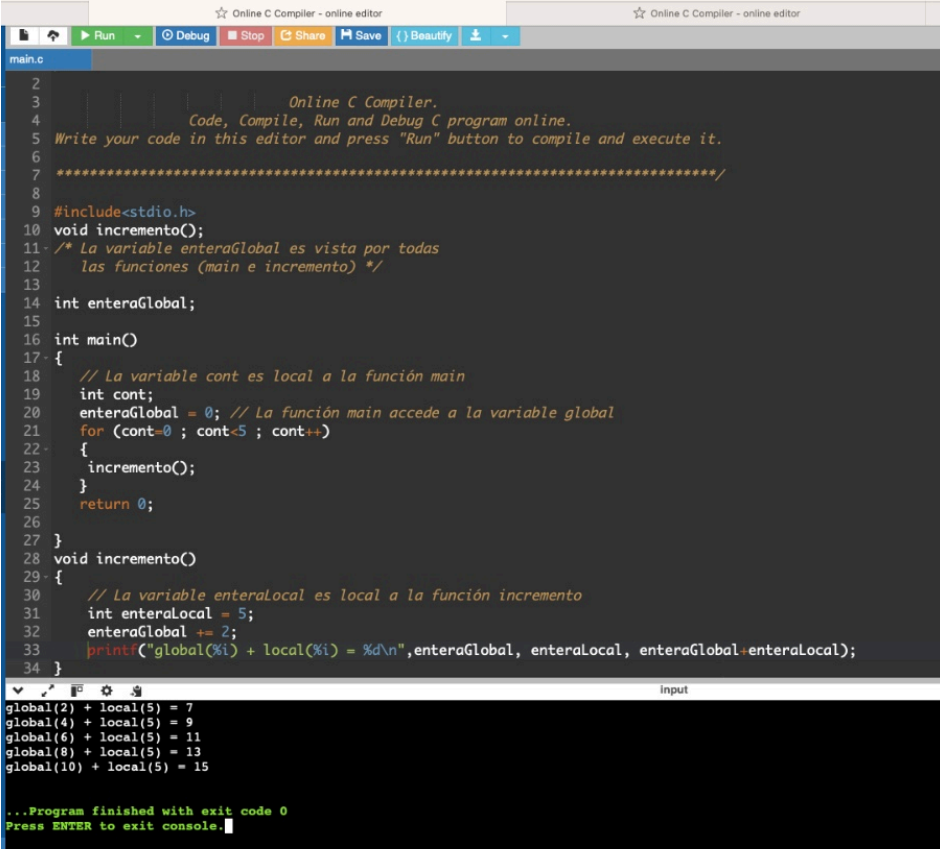
20

...Program finished with exit code 0
Press ENTER to exit console.

Figura 3: Código variables globales

Programa 4.c

Este programa contiene dos funciones: la función main y la función incremento. La función main manda llamar a la función incremento dentro de un ciclo for. La función incremento aumenta el valor de la variable enteraGlobal cada vez que es invocada.



```
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
main.c
Online C Compiler - online editor
Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.
*****/
#include<stdio.h>
void incremento();
/* La variable enteraGlobal es vista por todas
las funciones (main e incremento) */
int enteraGlobal;
int main()
{
    // La variable cont es local a la función main
    int cont;
    enteraGlobal = 0; // La función main accede a la variable global
    for (cont=0 ; cont<5 ; cont++)
    {
        incremento();
    }
    return 0;
}
void incremento()
{
    // La variable enteraLocal es local a la función incremento
    int enteraLocal = 5;
    enteraGlobal += 2;
    printf("global(%i) + local(%i) = %d\n",enteraGlobal, enteraLocal, enteraGlobal+enteraLocal);
}
```

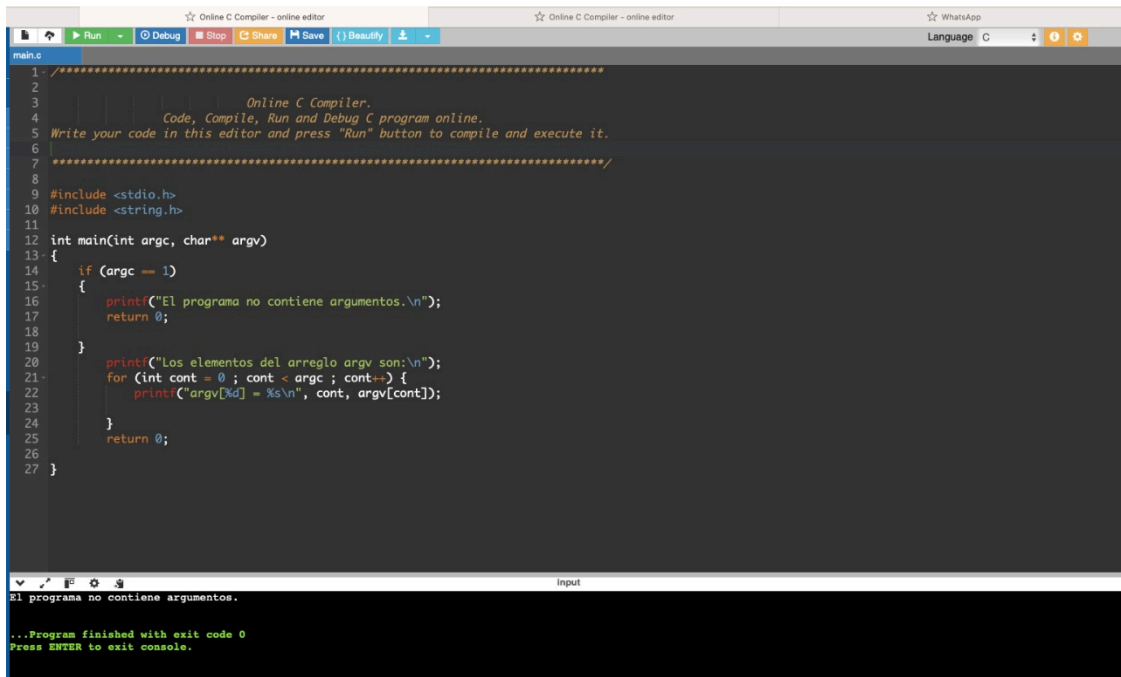
global(2) + local(5) = 7
global(4) + local(5) = 9
global(6) + local(5) = 11
global(8) + local(5) = 13
global(10) + local(5) = 15

...Program finished with exit code 0
Press ENTER to exit console.

Figura 4: Código de ámbitos de las variables

Programa 5.c

Este programa muestra los argumentos enviados al ejecutarlo.

The image shows a web-based C compiler interface. At the top, there are tabs for 'Online C Compiler - online editor' and 'WhatsApp'. Below the tabs is a toolbar with buttons for 'Run', 'Debug', 'Stop', 'Share', 'Save', and 'Beautify'. The main area is a code editor with a dark background, showing a C program. The program starts with a multi-line comment describing the compiler. It includes `<stdio.h>` and `<string.h>`. The `main` function takes `argc` and `argv` as arguments. It checks if `argc` is 1; if so, it prints 'El programa no contiene argumentos.' and returns 0. Otherwise, it prints the number of arguments and then iterates through `argv` to print each argument. The bottom of the interface shows the output of the program, which matches the first print statement, and a message indicating the program finished with exit code 0.

```
1: // *****  
2: // Online C Compiler.  
3: // Code, Compile, Run and Debug C program online.  
4: // Write your code in this editor and press "Run" button to compile and execute it.  
5: // *****  
6: //  
7: //  
8: //  
9: #include <stdio.h>  
10: #include <string.h>  
11:  
12: int main(int argc, char** argv)  
13: {  
14:     if (argc == 1)  
15:     {  
16:         printf("El programa no contiene argumentos.\n");  
17:         return 0;  
18:     }  
19:  
20:     printf("Los elementos del arreglo argv son:\n");  
21:     for (int cont = 0 ; cont < argc ; cont++) {  
22:         printf("argv[%d] = %s\n", cont, argv[cont]);  
23:     }  
24:     return 0;  
25: }  
26:  
27: }
```

El programa no contiene argumentos.

...Program finished with exit code 0
Press ENTER to exit console.

Figura 5: Código con argumentos en la función main

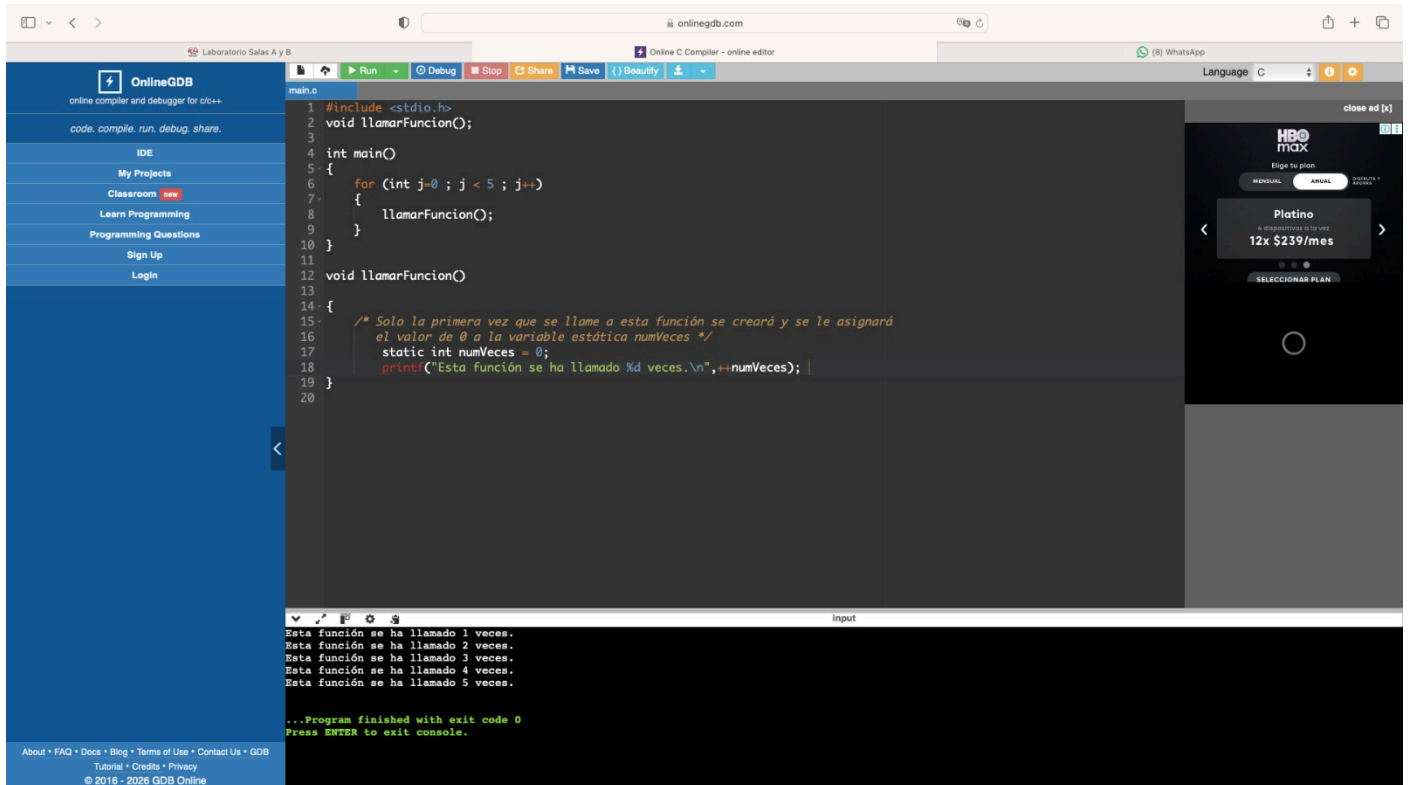
Programa 6.c

Este programa contiene dos funciones: la función `main` y la función `llamarFuncion`. La función `main` manda llamar a la función `llamarFuncion` dentro de un ciclo `for`. La función `llamarFuncion` crea una variable estática e imprime su valor.

La variable `numVeces` tiene un alcance local dentro de la función `llamarFuncion()`, lo que significa que solo puede ser utilizada dentro de esa función. Sin embargo, al ser declarada como `static`, su valor no se destruye cuando la función termina.

Una variable `static` permanece en memoria durante toda la ejecución del programa. Aunque `numVeces` solo es accesible desde `llamarFuncion()`, su valor se conserva entre diferentes llamadas a la función.

La variable `numVeces` se inicializa a 0 solo la primera vez que se ejecuta la función `llamarFuncion()`. En las siguientes llamadas, la línea `static int numVeces = 0;` no se ejecuta nuevamente porque la variable ya existe en memoria



```
1 #include <stdio.h>
2 void llamarFuncion();
3
4 int main()
5 {
6     for (int j=0 ; j < 5 ; j++)
7     {
8         llamarFuncion();
9     }
10 }
11
12 void llamarFuncion()
13 {
14     /* Solo la primera vez que se llame a esta función se creará y se le asignará
15     el valor de 0 a la variable estática numVeces */
16     static int numVeces = 0;
17     printf("Esta función se ha llamado %d veces.\n", ++numVeces);
18 }
19
20
```

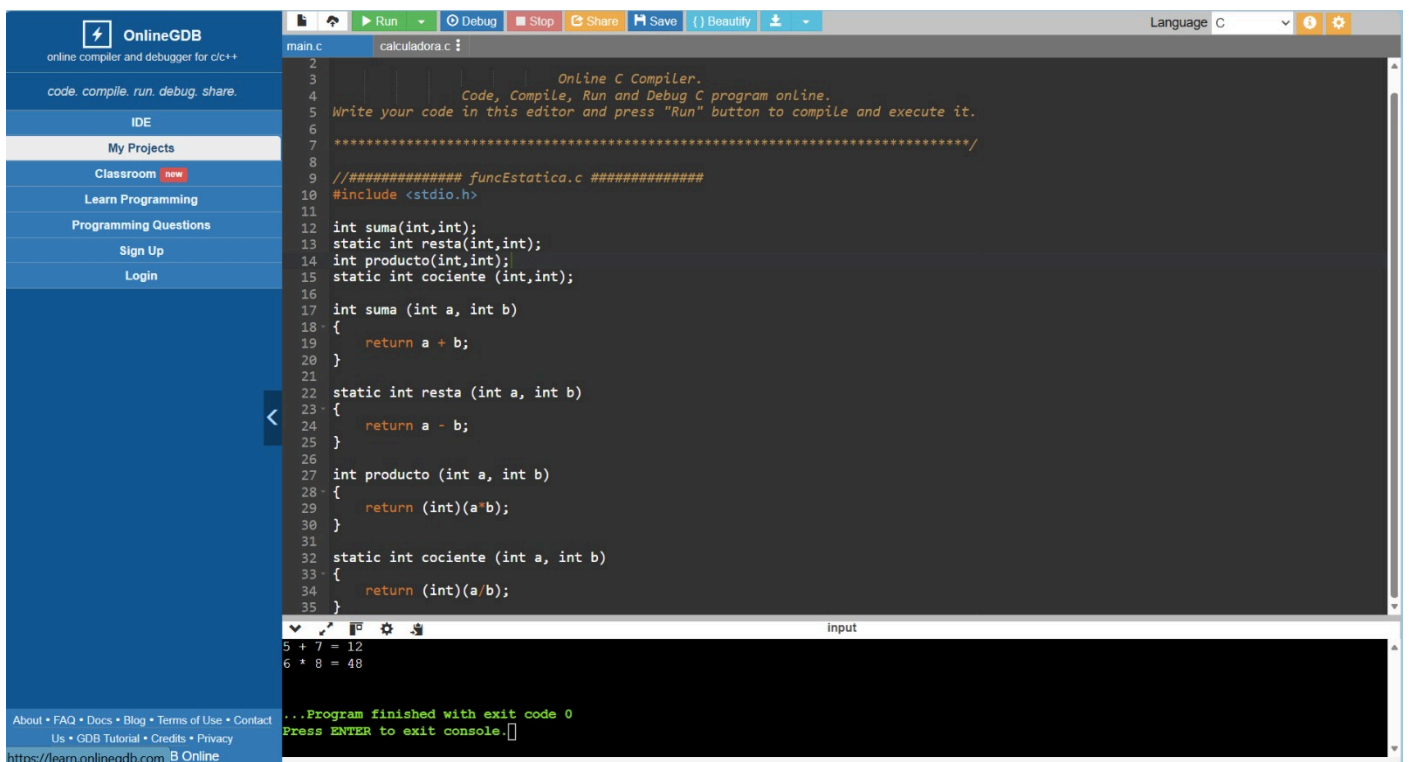
Output:

```
Esta función se ha llamado 1 veces.
Esta función se ha llamado 2 veces.
Esta función se ha llamado 3 veces.
Esta función se ha llamado 4 veces.
Esta función se ha llamado 5 veces.
...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 6: Código con variable estática

Calculadora.c

El programa funcEstatica.c contiene las funciones de una calculadora básica: suma, resta, producto y cociente.



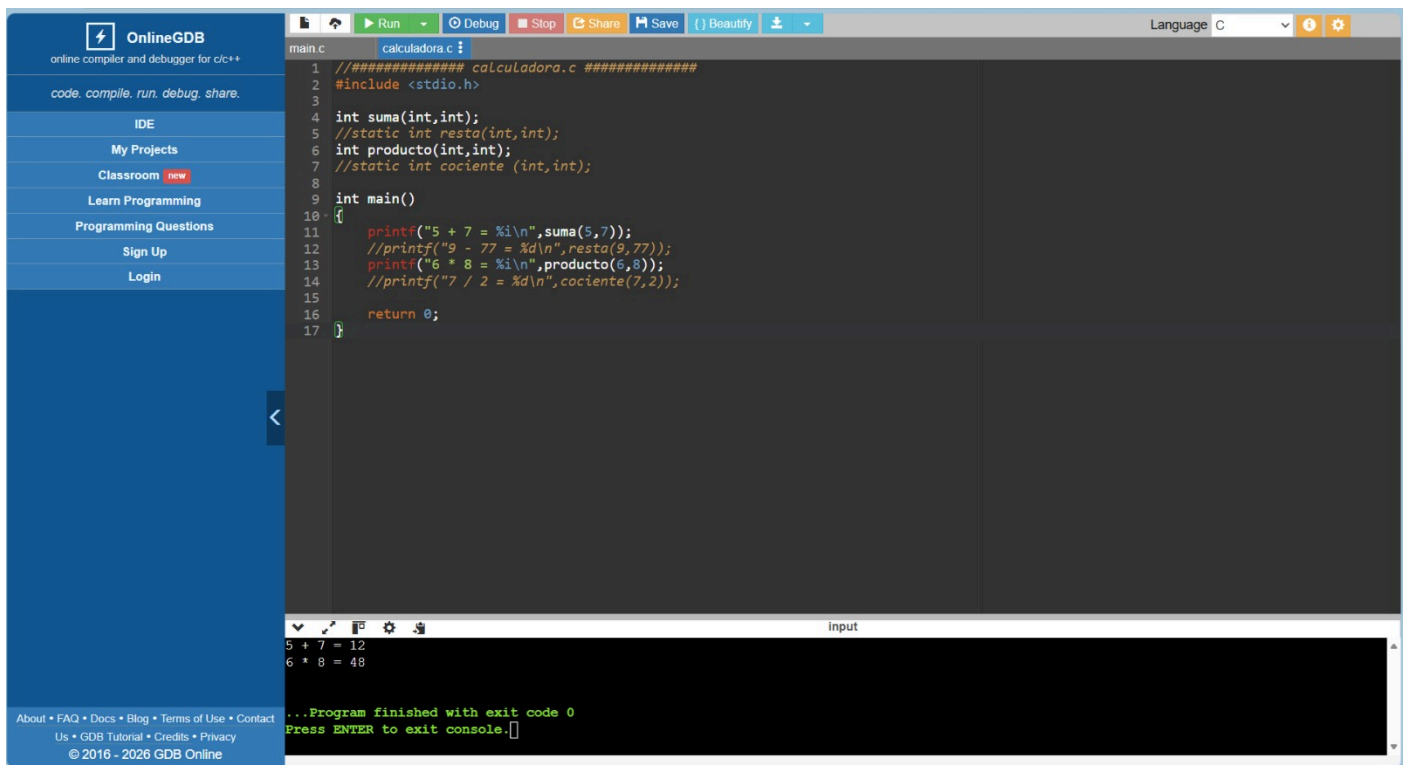
```
1
2
3 Online C Compiler.
4 Code, Compile, Run and Debug C program online.
5 Write your code in this editor and press "Run" button to compile and execute it.
6
7 =====/
8
9 //##### funcEstatica.c #####
10 #include <stdio.h>
11
12 int suma(int,int);
13 static int resta(int,int);
14 int producto(int,int);
15 static int cociente (int,int);
16
17 int suma (int a, int b)
18 {
19     return a + b;
20 }
21
22 static int resta (int a, int b)
23 {
24     return a - b;
25 }
26
27 int producto (int a, int b)
28 {
29     return (int)(a*b);
30 }
31
32 static int cociente (int a, int b)
33 {
34     return (int)(a/b);
35 }
36
```

Input:

```
5 + 7 = 12
6 * 8 = 48
...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 7 : Función estática

El programa calculadora.c contiene el método principal, el cual invoca a las funciones del archivo funcEstatica.c.



```
1 //##### calculadora.c #####
2 #include <stdio.h>
3
4 int suma(int,int);
5 //static int resta(int,int);
6 int producto(int,int);
7 //static int cociente (int,int);
8
9 int main()
10 {
11     printf("5 + 7 = %i\n",suma(5,7));
12     //printf("9 - 77 = %d\n",resta(9,77));
13     printf("6 * 8 = %i\n",producto(6,8));
14     //printf("7 / 2 = %d\n",cociente(7,2));
15
16     return 0;
17 }
```

Input

```
5 + 7 = 12
6 * 8 = 48
...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 8 : Calculadora

Cuando se compilan los dos archivos al mismo tiempo con la línea de comandos, las funciones suma y producto son accesibles desde el archivo calculadora y, por tanto, se genera el código ejecutable. Si se quitan los comentarios y se intenta compilar los archivos se enviará un error, debido a que las funciones son estáticas y no pueden ser accedidas fuera del archivo funcEstaticas.c.

Análisis de resultados:

Programa 1.c

En este programa se comprobó el uso de funciones en el lenguaje C mediante la interacción entre la función principal main y la función imprimir. El programa permitió observar cómo una función puede recibir parámetros, en este caso una cadena de caracteres, para posteriormente recorrerla e imprimirla en orden inverso. Además, se verificó el uso correcto de la biblioteca string.h y de la función strlen, la cual permitió obtener la longitud de la cadena para controlar adecuadamente el recorrido del ciclo.

Durante la ejecución se obtuvo correctamente la impresión invertida de la cadena almacenada en el arreglo de caracteres, demostrando que la comunicación entre funciones se realizó de manera adecuada. Asimismo, este ejercicio permitió identificar la importancia de declarar previamente los

prototipos de las funciones para evitar errores durante la compilación y facilitar la lectura del programa. También se observó que el uso de funciones ayuda a dividir un problema grande en tareas más pequeñas y específicas, haciendo que el código sea más organizado, reutilizable y sencillo de mantener.

Por otra parte, el programa permitió reforzar el manejo de arreglos de caracteres y el uso de ciclos para recorrer cadenas. Se concluyó que la modularización del código mediante funciones es una herramienta fundamental en programación, ya que mejora significativamente la claridad y eficiencia en el desarrollo de programas.

Programa 2.c

En este programa se analizó el funcionamiento de las variables locales dentro de una función. Se observó que las variables `x`, `y` y `z` solo existen dentro de la función `sumar`, por lo que no pueden ser utilizadas fuera de ella. Asimismo, se comprobó que las variables locales se crean al momento de ejecutar la función y desaparecen al finalizar ésta, limitando su alcance únicamente al bloque donde fueron declaradas.

El programa ejecutó correctamente la suma de los valores asignados a las variables y mostró el resultado esperado en pantalla, confirmando el correcto funcionamiento de la función. Este ejercicio permitió comprender cómo las variables locales ayudan a proteger la información dentro de cada función y evitan conflictos con variables declaradas en otras partes del programa. Además, se observó que las variables locales permiten que cada función trabaje de manera independiente, favoreciendo una mejor organización del código.

También se reforzó el concepto de modularidad en programación, ya que la función `sumar` realiza una tarea específica sin depender de otros elementos externos. Esto permitió comprender que el uso adecuado de variables locales facilita el mantenimiento y depuración de los programas, especialmente en proyectos de mayor tamaño.

Programa 3.c

El desarrollo de este programa permitió identificar el funcionamiento de las variables globales en C. La variable `resultado` fue declarada fuera de las funciones, lo que permitió que tanto `main` como `multiplicar` pudieran acceder y modificar su valor sin necesidad de enviarla como parámetro. Se comprobó que las variables globales permanecen disponibles durante toda la ejecución del programa y que su valor inicial por defecto es cero.

Durante la ejecución, la función `multiplicar` realizó la operación correspondiente y almacenó el resultado en la variable global. Posteriormente, la función `main` mostró correctamente el valor obtenido en pantalla. Esto permitió observar cómo una variable global puede ser compartida entre distintas funciones, facilitando el intercambio de información dentro del programa.

Sin embargo, el ejercicio también permitió comprender que el uso excesivo de variables globales puede provocar desorden o errores en programas grandes, ya que cualquier función puede modificar su contenido. Por ello, se concluyó que las variables globales son útiles cuando varias

funciones necesitan compartir información, pero deben utilizarse con precaución para evitar problemas en el control y seguimiento de los datos.

Programa 4.c

En este programa se analizaron simultáneamente las variables locales y globales para comprender sus diferencias de alcance, duración y comportamiento durante la ejecución. La función `incremento` modificó el valor de la variable global `enteraGlobal` cada vez que fue llamada desde el ciclo `for` de la función `main`. Además, dentro de la misma función se declaró la variable local `enteraLocal`, la cual únicamente podía utilizarse dentro de ese bloque de código.

Los resultados mostraron que la variable global conservó y actualizó su valor entre llamadas consecutivas, incrementándose correctamente en cada iteración del ciclo. En cambio, la variable local mantuvo siempre el mismo valor inicial debido a que era creada nuevamente cada vez que se ejecutaba la función. Esto permitió comprender claramente la diferencia entre una variable global, que conserva y comparte su información durante toda la ejecución del programa, y una variable local, cuyo tiempo de vida es temporal y limitado a la función donde fue creada.

Además, el programa permitió reforzar el uso de ciclos y funciones dentro de un mismo código, observando cómo varias estructuras pueden trabajar conjuntamente para resolver una tarea. Se concluyó que comprender el ámbito de las variables es fundamental para evitar errores lógicos y asegurar un correcto manejo de la información dentro de un programa.

Programa 5.c

Este programa permitió analizar el uso de argumentos en la función `main`, utilizando los parámetros `argc` y `argv`. Se comprobó que estos argumentos permiten recibir información desde la línea de comandos al momento de ejecutar el programa, proporcionando una manera dinámica de ingresar datos sin necesidad de capturarlos directamente desde el teclado.

Durante la ejecución, el programa mostró correctamente el número de argumentos introducidos y posteriormente imprimió cada elemento almacenado en el arreglo `argv`, incluyendo el nombre del programa ejecutable. Esto ayudó a comprender cómo los programas en C pueden interactuar con información proporcionada externamente y cómo los argumentos pueden utilizarse para desarrollar aplicaciones más flexibles y automatizadas.

Asimismo, el ejercicio permitió reforzar el manejo de arreglos de cadenas y ciclos de repetición para recorrer los elementos recibidos desde la línea de comandos. También se comprendió la utilidad de este tipo de argumentos en aplicaciones reales, donde es común enviar parámetros al momento de ejecutar un programa. Finalmente, se concluyó que el uso de argumentos en `main` amplía considerablemente las posibilidades de interacción y personalización de los programas.

Programa 6.c

En este ejercicio se estudió el comportamiento de las variables estáticas mediante la variable `numVeces`. Aunque dicha variable fue declarada dentro de la función `llamarFuncion`, su valor permaneció almacenado entre diferentes llamadas debido al uso de la palabra reservada `static`. Esto permitió observar que una variable estática conserva su información durante toda la ejecución del programa, aun cuando su alcance siga siendo local.

Durante las diferentes iteraciones del ciclo `for`, el programa mostró cómo el contador aumentó progresivamente en cada llamada a la función, en lugar de reiniciarse en cero. Este comportamiento permitió diferenciar claramente una variable local común de una variable local estática. Mientras una variable local tradicional pierde su valor al finalizar la función, la variable estática conserva el último valor almacenado para reutilizarlo posteriormente.

Además, el ejercicio ayudó a comprender la utilidad de las variables estáticas para llevar registros, contar eventos o conservar información entre múltiples ejecuciones de una función. Se concluyó que las variables estáticas representan una herramienta importante en programación cuando se requiere mantener datos persistentes sin utilizar variables globales.

Calculadora.c

El análisis de este programa permitió comprender el uso de funciones estáticas dentro de distintos archivos en C y la manera en que interactúan las unidades de compilación. El archivo `funcEstatica.c` contenía las funciones matemáticas de la calculadora, mientras que `calculadora.c` incluía la función principal encargada de invocar algunas de ellas.

Durante la compilación conjunta de ambos archivos se observó que las funciones `suma` y `producto` pudieron utilizarse correctamente desde `calculadora.c`, debido a que fueron declaradas como funciones normales. En cambio, las funciones `resta` y `cociente` no podían ser llamadas desde otro archivo porque fueron declaradas con el modificador `static`, restringiendo su acceso únicamente al archivo donde fueron definidas.

Esto permitió comprender que las funciones estáticas ayudan a proteger y limitar el acceso al código interno de un archivo, mejorando la organización y seguridad de los programas. Asimismo, se comprobó que si se eliminaban los comentarios de las líneas que llamaban a las funciones estáticas, el compilador generaba errores, demostrando el correcto funcionamiento del atributo `static`.

Finalmente, este ejercicio permitió entender cómo se realiza la compilación de múltiples archivos en lenguaje C y cómo se pueden dividir los programas en módulos independientes. Esto facilita el desarrollo de proyectos más grandes, organizados y fáciles de mantener, ya que cada archivo puede encargarse de tareas específicas dentro del programa general.

Conclusión:

Macay Martínez David:

Tras el desarrollo de la práctica, se concluyó que los arreglos bidimensionales son herramientas de almacenamiento importantes para programar, ya que proporcionan una estructura lógica que se alinea con la organización de diversos datos en la ingeniería. A comparación de los arreglos unidimensionales, el uso de matrices facilita la representación de sistemas complejos donde la información depende de 2 variables.

A través del desarrollo de los programas, se demostró que el manejo de 2 índices es la base del control de estructuras. Se comprendió que el primer índice maneja el desplazamiento de filas y el segundo el horizontal, este orden es importante de respetar para implementar ciclos anidados.

Finalmente, se visualizaron los datos ingresados en pantalla en forma de tabla, a pesar de que el compilador los almacena de forma lineal. La elaboración de esta práctica permite reforzar habilidades para decidir en que tipo de situaciones una estructura bidimensional es la solución más óptima.

Nolasco Ramírez Sofia:

En el desarrollo de esta práctica se lograron cumplir satisfactoriamente los objetivos planteados, ya que se elaboraron distintos programas en lenguaje C enfocados en el uso de funciones, manejo de variables y aplicación del modificador `static`. A través de los ejercicios realizados se comprendió la importancia de dividir un problema en funciones específicas, permitiendo una mejor organización, claridad y reutilización del código.

Asimismo, se identificaron las diferencias entre variables locales, globales y estáticas, observando su comportamiento y alcance durante la ejecución de los programas. También se reforzó el uso de prototipos de funciones, argumentos en la función `main` y la compilación de múltiples archivos, comprendiendo cómo las funciones estáticas restringen el acceso únicamente al archivo donde fueron definidas.

Esta práctica permitió fortalecer los conocimientos fundamentales de programación en C, desarrollando una mejor comprensión sobre la modularidad, el manejo adecuado de variables y la estructuración correcta de programas para crear códigos más eficientes, ordenados y fáciles de mantener.

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.